

1480. 一维数组的动态和

已解答

简单 相关标签 相关企业 提示 Aa

给你一个数组 `nums`。数组「动态和」的计算公式为：`runningSum[i] = sum(nums[0]..nums[i])`。

请返回 `nums` 的动态和。

示例 1:

输入: `nums = [1,2,3,4]`

C++ 智能模式

```
1 class Solution {
2 public:
3     vector<int> runningSum(vector<int>& nums) {
4         int n = nums.size();
5         for (int i = 1; i < n; i++) {
6             nums[i] += nums[i - 1];
7         }
8         return nums;
9     }
10 };
```

2574. 左右元素和的差值

已解答

简单 相关标签 相关企业 提示 Aa

给你一个下标从 0 开始的长度为 `n` 的整数数组 `nums`。

定义两个数组 `leftSum` 和 `rightSum`，其中：

- `leftSum[i]` 是数组 `nums` 中下标 `i` 左侧元素之和。如果不存在对应的元素，`leftSum[i] = 0`。
- `rightSum[i]` 是数组 `nums` 中下标 `i` 右侧元素之和。如果不存在对应的元素，`rightSum[i] = 0`。

返回长度为 `n` 数组 `answer`，其中 `answer[i] = |leftSum[i] - rightSum[i]|`。

示例 1:

输入: `nums = [10,4,8,3]`

输出: `[15,1,11,22]`

解释: 数组 `leftSum` 为 `[0,10,14,22]`，且数组 `rightSum` 为 `[15,11,3,0]`。

C++ 智能模式

```
1 class Solution {
2 public:
3     vector<int> leftRightDifference(vector<int>& nums) {
4         int n = nums.size();
5         vector<int> ans(n);
6
7         int leftSum = 0;
8         for (int i = 0; i < n; ++i) {
9             ans[i] = leftSum;
10            leftSum += nums[i];
11        }
12
13        int rightSum = 0;
14        for (int i = n - 1; i >= 0; --i) {
15            ans[i] = abs(ans[i] - rightSum);
16            rightSum += nums[i];
17        }
18
19        return ans;
20    }
21 };
```